

Automated Deployment Pipeline - Live Test Run

CI/CD Pipeline: Live Deployment Simulation

Context / Setup Before Simulation

To validate that the CI/CD pipeline works correctly end-to-end, a live simulation was performed using two separate environments:

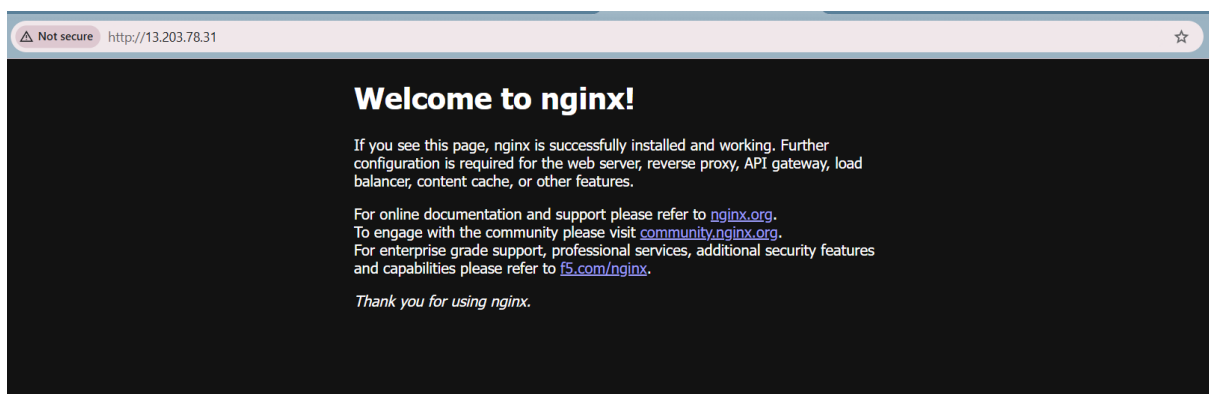
Environment	Role	Details
EC2 Production	Live production server	IP: 13.203.78.31 currently running the older, bugged version of the code (pre-fix), representing the real-world starting state before automated deployment
EC2 Developer	Development machine	Contains the fixed application code (Dockerfile, docker-compose.yml, nginx.conf fixes) along with the deploy.yml GitHub Actions workflow, ready to be pushed to the repository

Baseline State — Before Deployment

Before triggering the pipeline, the production server's current state was captured to establish a clear before/after comparison.

Old Code (Pre-Fix):

Visiting the production IP directly showed Nginx's default welcome page, confirming the server was still running the original, unfixed configuration.



Containers Running on Old Code:

docker ps on EC2 Production confirmed two containers active nginx:alpine and the backend both Up, but serving the broken configuration rather than the working chat application.

```
ubuntu@ip-172-31-7-254:~/Real-Time-WebSocket-Application$ docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS                               NA
MES
4da911235970   nginx:alpine                        "/docker-entrypoint..." 22 minutes ago Up 22 minutes 0.0.0.0:80->80/tcp, [::]:80->80/tcp ch
22870faaa6f9   real-time-websocket-application--backend "uvicorn main:app --..." 22 minutes ago Up 22 minutes 8000/tcp                               ch
ubuntu@ip-172-31-7-254:~/Real-Time-WebSocket-Application$ curl ifconfig.me
13.203.78.31ubuntu@ip-172-31-7-254:~/Real-Time-WebSocket-Application$
```

Step: Pushing Fixed Code from EC2 Developer

With the baseline confirmed, the corrected code was pushed from the developer environment to trigger the pipeline.

Command executed (on EC2 Developer):

git push origin main

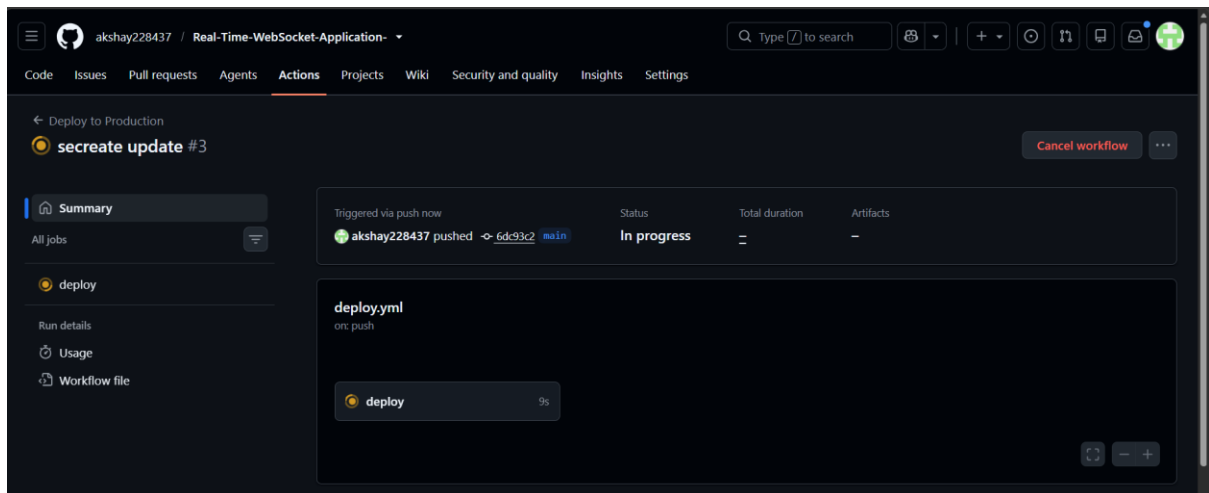
Purpose:

This pushes the corrected Dockerfile, docker-compose.yml, nginx.conf, and the deploy.yml GitHub Actions workflow from the developer environment to the main branch of the GitHub repository.

```
1 file changed, 19 insertions(+), 19 deletions(-)
...
● ubuntu@ip-172-31-6-197:~/Real-Time-WebSocket-Application$ git push origin main
Enumerating objects: 9, done.
Counting objects: 100% (9/9), done.
Delta compression using up to 2 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (5/5), 509 bytes | 509.00 KiB/s, done.
Total 5 (delta 2), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (2/2), completed with 2 local objects.
To github.com:akshay228437/Real-Time-WebSocket-Application-.git
   b0770a9..6d15507  main -> main
● ubuntu@ip-172-31-6-197:~/Real-Time-WebSocket-Application$
```

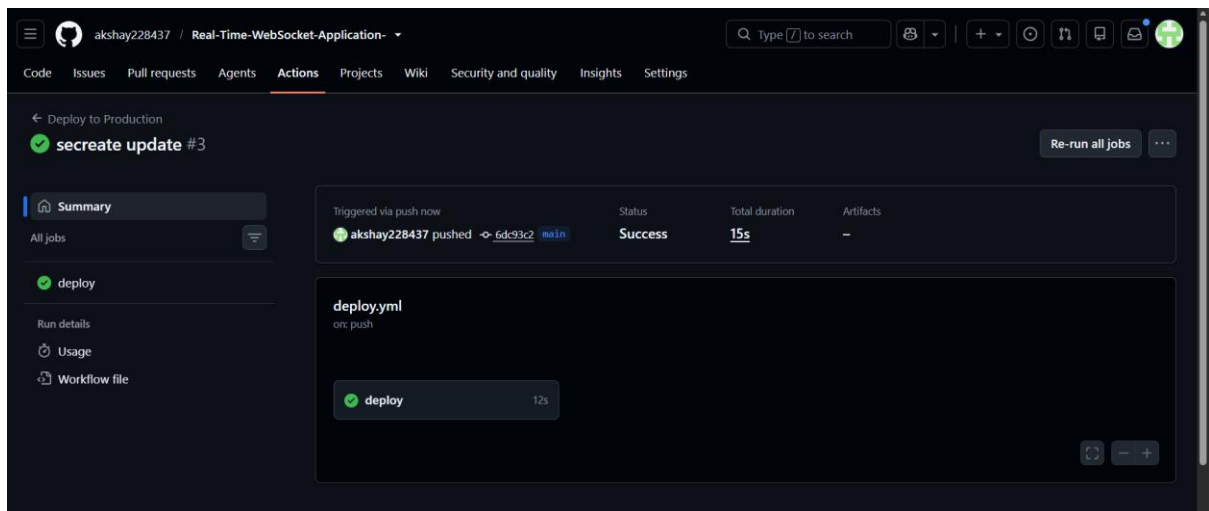
Pipeline Triggered Automatically

GitHub detected the push to main and automatically started the GitHub Actions workflow — no manual step was needed to kick off the pipeline from this point onward. The Actions tab showed the run "secreate update #3" in progress, executing the deploy job defined in deploy.yml.



Successful Completion

The workflow completed successfully in 15 seconds, with the deploy job showing a green checkmark. This confirms the GitHub Actions runner successfully authenticated to the production server via SSH, pulled the latest code, and rebuilt the containers — entirely without manual intervention.



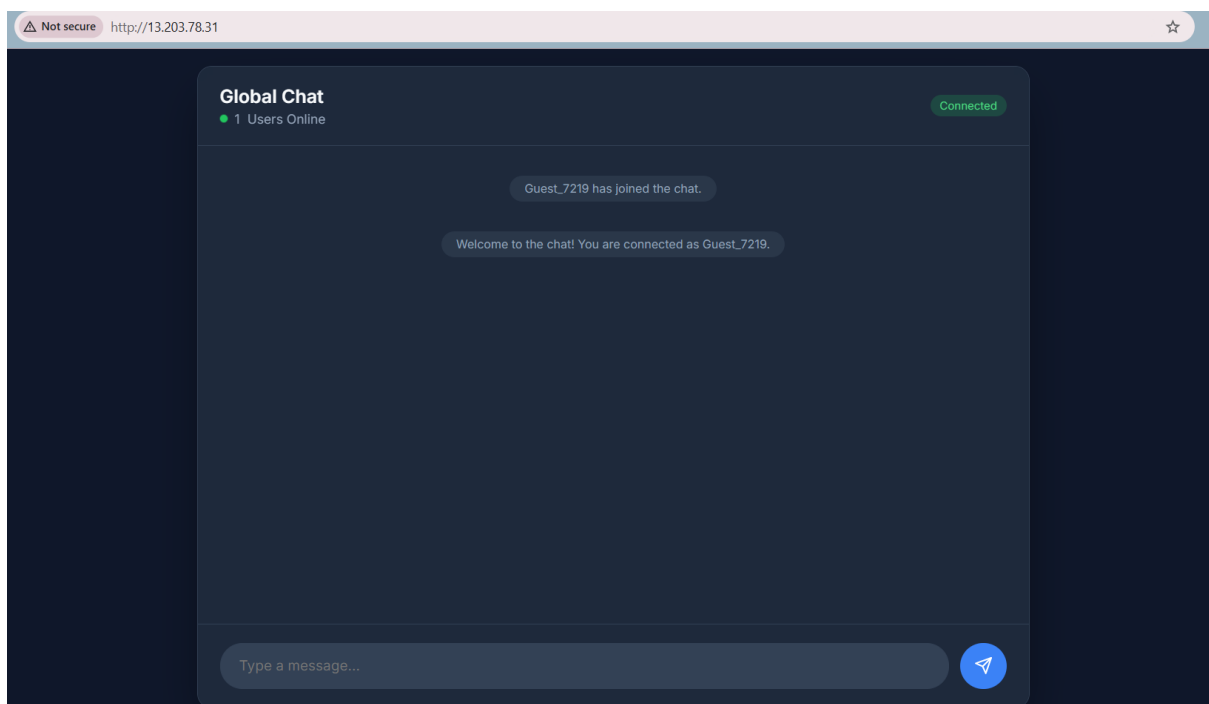
Containers Deployed with New Code

Post-deployment verification on EC2 Production confirmed both containers were rebuilt and running with fresh container IDs and updated "Up" durations matching the deployment timestamp, confirming the automated rebuild had actually taken effect rather than the old containers simply continuing to run.

```
ubuntu@ip-172-31-7-254:~/Real-Time-WebSocket-Application$ curl ifconfig.me
13.203.78.31ubuntu@ip-172-31-7-254:~/Real-Time-WebSocket-Application$ docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS                NA
MES
b04ff4cb08ce   nginx:alpine                       "/docker-entrypoint..." 38 seconds ago Up 37 seconds   0.0.0.0:80->80/tcp, [::]:80->80/tcp  ch
1b87a889e81c   real-time-websocket-application--backend "uvicorn main:app --..." 38 seconds ago Up 37 seconds   8000/tcp            ch
at-backend
ubuntu@ip-172-31-7-254:~/Real-Time-WebSocket-Application$ |
```

Fixed Application Verified Live

Reloading **http://13.203.78.31** in the browser now showed the working chat application interface instead of the Nginx default page, with the connection status showing "Connected" and the user successfully joined as a guest.



Multi-Browser-Tab Real-Time Verification

To confirm real-time multi-user functionality, two separate browser tabs were opened simultaneously against the same production IP. Messages sent from one tab appeared instantly in the other, with both sessions showing "2 Users Online" confirming the WebSocket connection through Nginx correctly supports concurrent multi-user real-time chat in the deployed production environment.

